📌 What is Matplotlib?

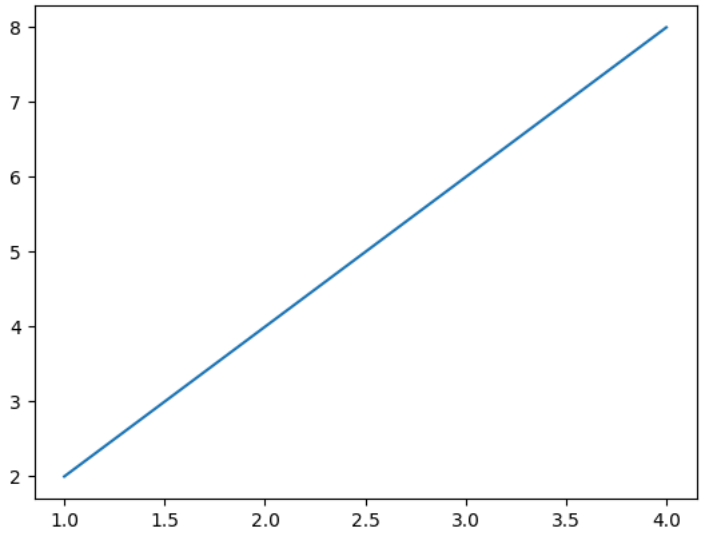
Matplotlib is a Python library used for data visualization.

It allows you to create static, interactive, and animated plots in Python.

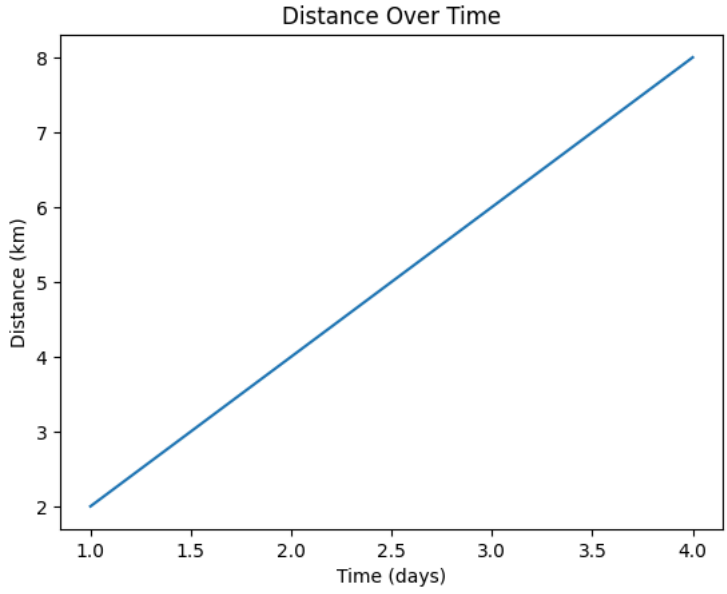
```
import matplotlib.pyplot as plt
```

```
#Basic Line Plot
x = [1, 2, 3, 4]
y = [2, 4, 6, 8]

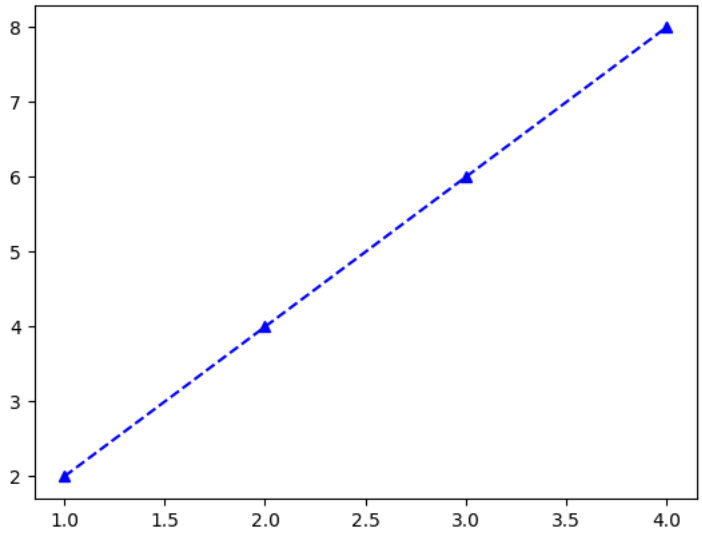
plt.plot(x, y)
plt.show()
```



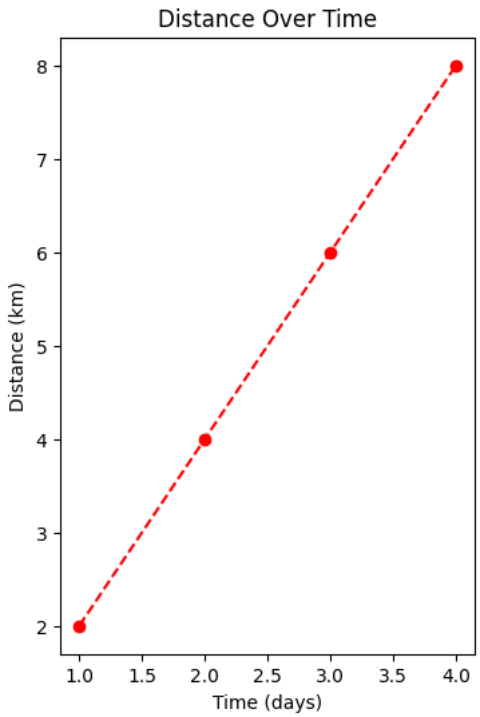

```
# Add Labels & Title
plt.plot(x, y)
plt.xlabel("Time (days)")
plt.ylabel("Distance (km)")
plt.title("Distance Over Time")
plt.show()
```




```
#Change Color, Line, & Marker
plt.plot(x, y, color="blue",linestyle='--', marker="^")
plt.show()
```

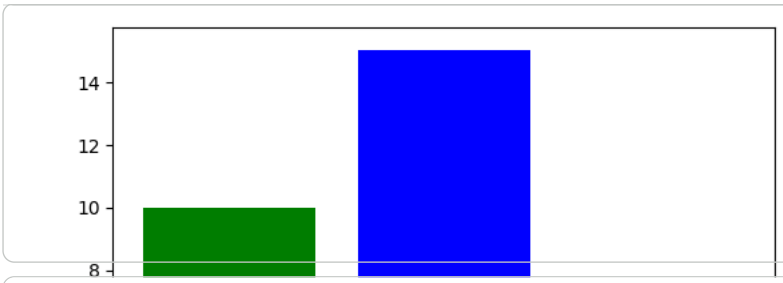



```
#Change Figure Size
plt.figure(figsize=(4,6))
plt.plot(x, y, color="red", linestyle="--", marker="o")
plt.xlabel("Time (days)")
plt.ylabel("Distance (km)")
plt.title("Distance Over Time")
plt.show()
```



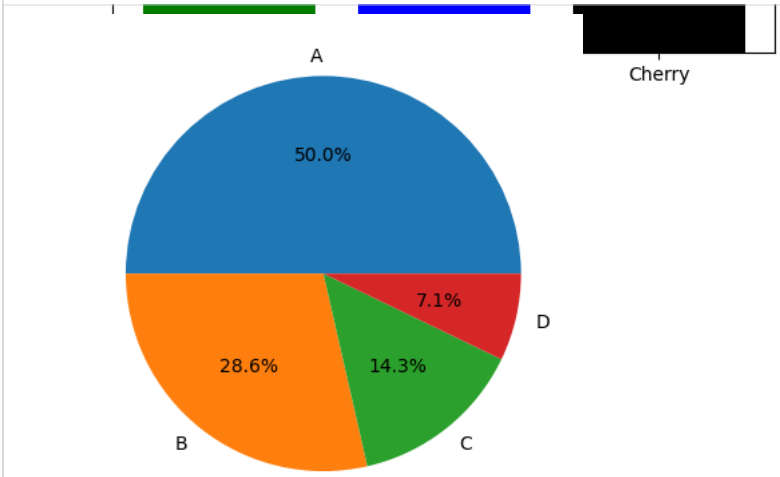

```
# Bar Chart
fruits = ["Apple", "Banana", "Cherry"]
sales = [10, 15, 7]

plt.bar(fruits, sales, color=["green","blue","black"])
plt.show()
```




```
#Pie Chart
sizes = [70, 40, 20, 10]
name = ["A", "B", "C", "D"]

plt.pie(sizes, labels=name, autopct="%1.1f%%")
plt.show()
```



autopct stands for "automatic percentage".

It controls how the percentages are displayed inside each slice of the pie chart.

In your case:

"%1.1f%%" is a format string.

% → special placeholder for string formatting.

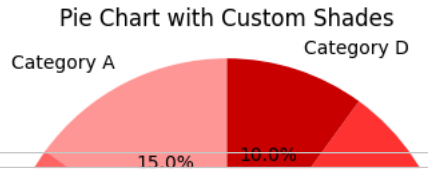
1.1f → means floating-point number with 1 digit before decimal and 1 digit after decimal.

%% → prints a literal % symbol.

So each slice will display the percentage with one decimal place, like 30.0%, 40.0%, etc.

```
sizes = [15, 30, 45, 10]
lab = ['Category A', 'Category B', 'Category C', 'Category D']
# Define custom colors with shades
col = ['#FF9999', '#FF6666', '#FF3333', '#CC0000'] # Shades of red

plt.pie(sizes, labels=lab, colors=col, autopct='%1.1f%%', startangle=90)
plt.axis('equal') # Equal aspect ratio ensures that pie is drawn as a circle.
plt.title('Pie Chart with Custom Shades')
plt.show()
```

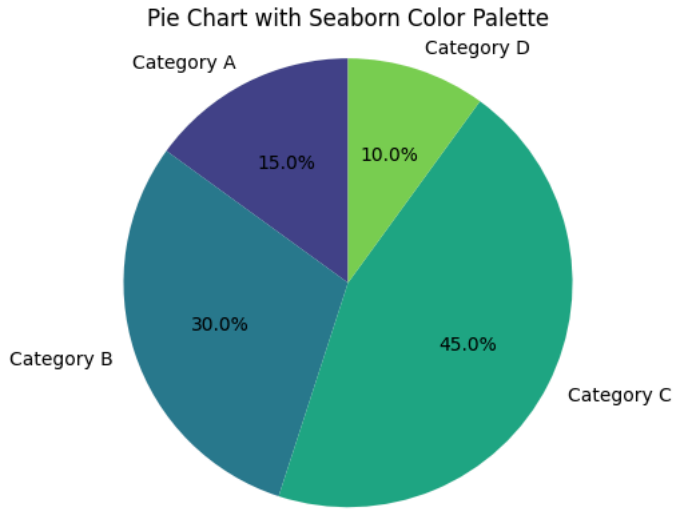



```
import matplotlib.pyplot as plt
import seaborn as sns

sizes = [15, 30, 45, 10]
lab = ['Category A', 'Category B', 'Category C', 'Category D']

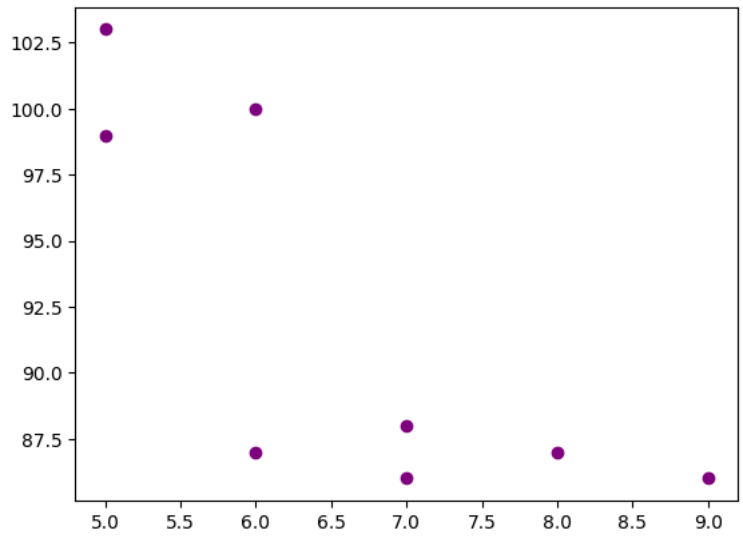
# Use a Seaborn color palette
colo = sns.color_palette('viridis', len(sizes)) # 'viridis' provides a gradient of shades

plt.pie(sizes, labels=lab, colors=colo, autopct='%1.1f%%', startangle=90)
plt.axis('equal')
plt.title('Pie Chart with Seaborn Color Palette')
plt.show()
```



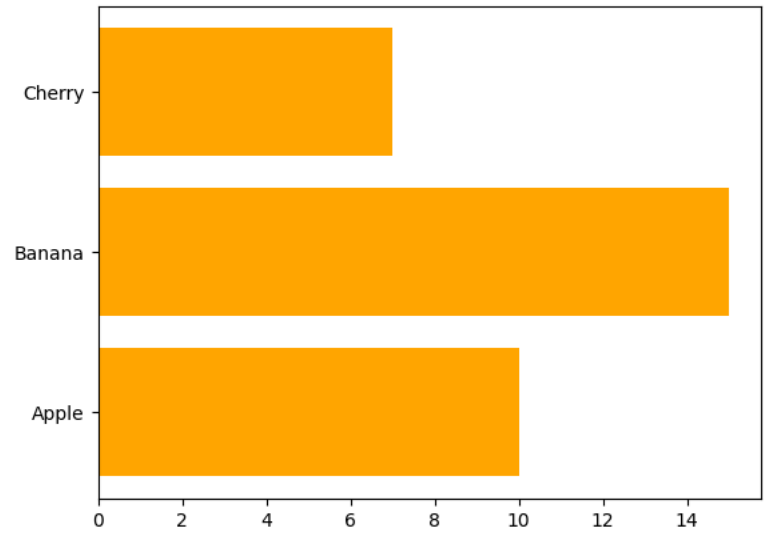

```
# Scatter Plot
x = [5, 7, 8, 7, 6, 9, 5, 6]
y = [99, 86, 87, 88, 100, 86, 103, 87]

plt.scatter(x, y, color="purple")
plt.show()
```




```
# Horizontal Bar Chart
fruits = ["Apple", "Banana", "Cherry"]
sales = [10, 15, 7]
```

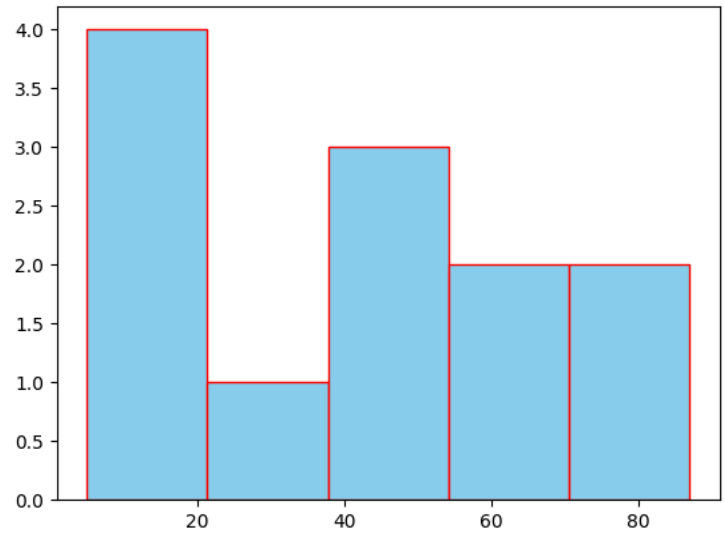
```
plt.barh(fruits, sales, color="orange")
plt.show()
```



Double-click (or enter) to edit

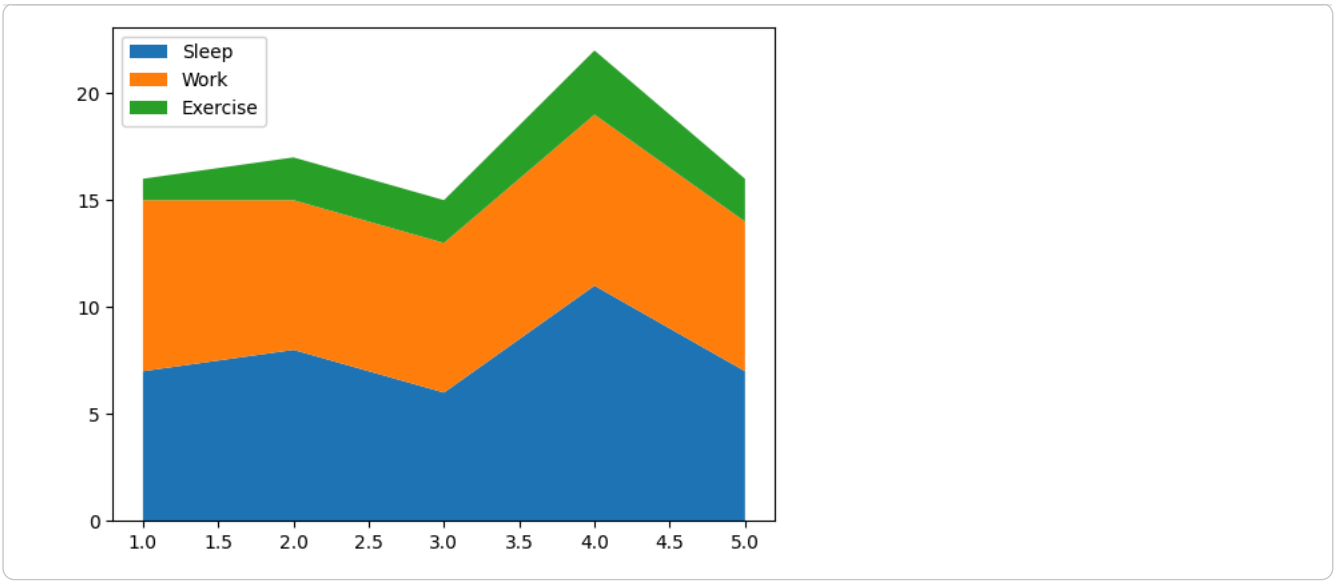
```
#Histogram
data = [22, 87, 5, 43, 56, 73, 55, 54, 11, 20, 51, 5]

plt.hist(data, bins=5, color="skyblue", edgecolor="red")
plt.show()
```




```
# stack plot
days = [1, 2, 3, 4, 5]
sleep = [7, 8, 6, 11, 7]
work = [8, 7, 7, 8, 7]
exercise = [1, 2, 2, 3, 2]

plt.stackplot(days, sleep, work, exercise, labels=["Sleep", "Work", "Exercise"])
plt.legend()
plt.show()
```



X-axis ⟶ Days (1 to 5)

Y-axis ⟶ Hours (stacked total of Sleep, Work, Exercise per day)

X-axis (days)

Values: 1, 2, 3, 4, 5

Represents days (Day 1 ⟶ Day 5).

Y-axis (stacked values)

Shows the cumulative total of sleep + work + exercise for each day.

Each category (Sleep, Work, Exercise) is stacked on top of the previous one.

Example for each day:

Day 1: Sleep = 7, Work = 8, Exercise = 1 Total = 16

Day 2: Sleep = 8, Work = 7, Exercise = 2 Total = 17

Day 3: Sleep = 6, Work = 7, Exercise = 2 Total = 15

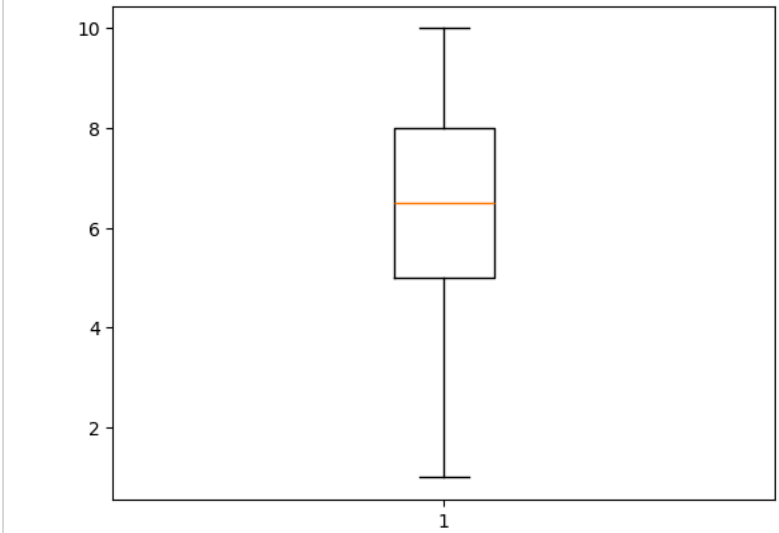
Day 4: Sleep = 11, Work = 8, Exercise = 3 Total = 22

Day 5: Sleep = 7, Work = 7, Exercise = 2 Total = 16

So the Y-axis range goes up to a bit more than 22 (the maximum stacked total).

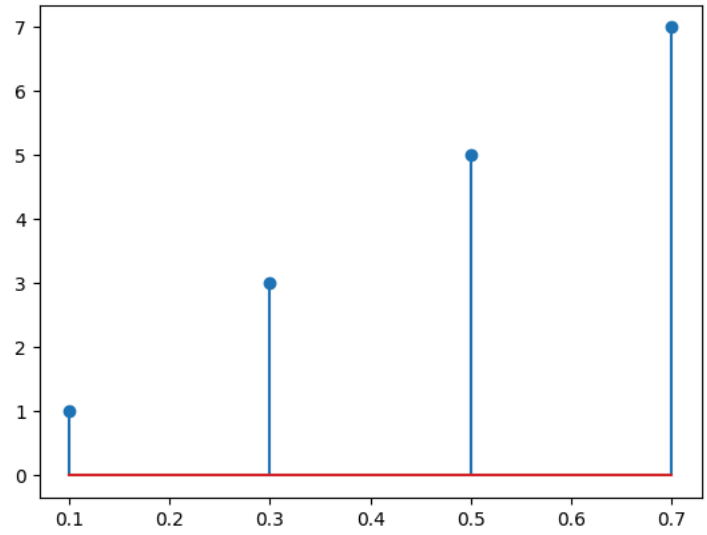
```
#Box Plot (Whisker Plot)
data = [7, 8, 5, 6, 9, 10, 4, 5, 6, 8, 7,1]

plt.boxplot(data)
plt.show()
```



```
#Stem Plot
x = [0.1, 0.3, 0.5, 0.7]
y = [1, 3, 5, 7]

plt.stem(x, y)
plt.show()
```

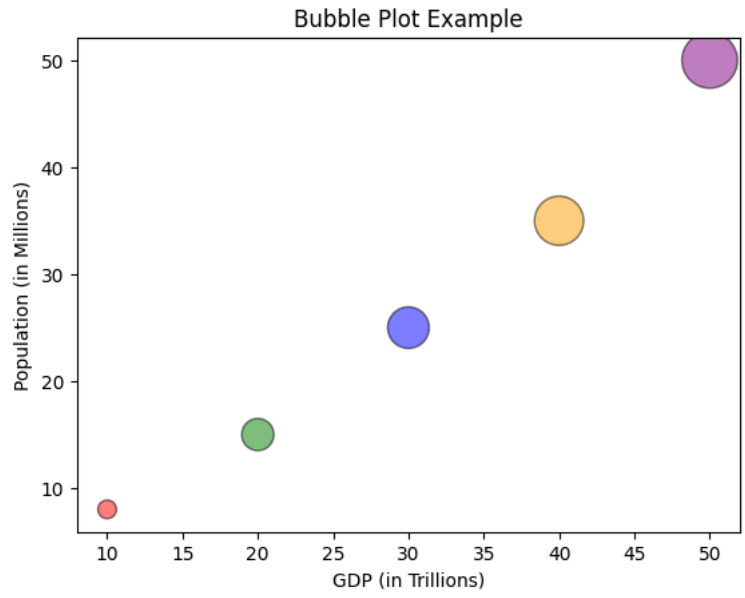



```
# Bubble plot
# Data
x = [10, 20, 30, 40, 50]   # X-axis values
y = [8, 15, 25, 35, 50]    # Y-axis values
sizes = [100, 300, 500, 700, 900]  # Bubble sizes
colors = ['red', 'green', 'blue', 'orange', 'purple']  # Colors

# Create bubble plot
plt.scatter(x, y, s=sizes, c=colors, alpha=0.5, edgecolors="black")

# Labels & title
plt.xlabel("GDP (in Trillions)")
plt.ylabel("Population (in Millions)")
plt.title("Bubble Plot Example")

plt.show()
```



Matplotlib Plots & Their Uses

Line Plot 📈

Use: Show trends or changes over time.

Example: Stock prices for 7 days, temperature during a week.

Bar Chart 📊

Use: Compare quantities across categories.

Example: Sales of different fruits.

Horizontal Bar Chart ↔️

Use: Compare categories with long names.

Example: Population of different countries.

Pie Chart 🥧

Use: Show proportions or percentages of a whole.

Example: Market share of mobile brands.

Scatter Plot 🟣

Use: Show relationship between two variables.

Example: Height vs. weight.

Histogram 📦

Use: Show distribution of data values.

Example: Age distribution of survey participants.

Box Plot (Whisker Plot) 📦📏

Use: Show median, spread, and outliers in data.

Example: Exam score distribution.

Stack Plot 📚

Use: Show how parts contribute to a whole over time.

Example: Hours spent on sleep, work, and exercise daily.

Stem Plot 🌱

Use: Show discrete data points clearly.

Example: Small experimental readings.

Bubble Plot 🫧 **(scatter with size)**

Use: Show relationship between variables and add size as an extra dimension.

Example: Countries' GDP vs. population with bubble size as internet usage.